

Unsupervised Methods for Time-Series

The performance of predictive models is undermined by major changes in the data. Recognizing these changes is the domain of unsupervised learning.

At the core of unsupervised learning is the extraction of structure from time-series, most importantly recognizing similarities between subsequences. This is the essence of anomaly detection, where we want to identify sequences that are notably different from the rest of the series.

Unsupervised Methods for Time-Series

An important challenge of working with time-series is recognizing the changes in the underlying processes. This is called change point detection (CPD) or drift detection.

Data changes over time, and it is crucial to recognize how much it changes. This is worth diving into, because the existence of change points and anomalous points are common problems with real-world applications.

Anomaly detection for Time-Series

In anomaly detection, we want to identify sequences that are notably different from the rest of the series. Anomalies or outliers can sometimes be the result of measurement error or noise, but they could indicate changes to behavior or aberrant behavior in the system under observation, which could require urgent action.

Definition: An outlier is a data point that deviates so significantly from other observations that it could have been generated by a different mechanism

Anomaly detection for Time-Series

In the simplest case, we can define an outlier as follows as the z-score of the observation x_i with respect to the distribution parameters:

$$\text{Outlier}(\hat{\mu}, \hat{\sigma}) = \{x: |x - \hat{x}| / \hat{\sigma} > \epsilon\}$$

The z-score measures the distance of each point from the moving average or sample mean, $\hat{x}$, in units of the moving or sample standard deviation $\hat{\sigma}$.

$\hat{\mu}$ and $\hat{\sigma}$ are the estimated mean and standard deviation of the timeseries, ϵ is a threshold dependent on the confidence interval that we are interested in, e.g., 2 or 1.96 are corresponding to a confidence interval of 95%. In this way, outliers are points that occur 5% or less of the time.

Anomaly detection for Time-Series

The Hampel filter (Hampel identifier)

$$HF(x) = \left\{ x: \frac{|x - \text{median}(x)|}{1.4826MAD(x)} > \varepsilon \right\}$$

where **MAD** is the **median absolute deviation**,

$$MAD(x) = \text{median}\{|x_i - \text{median}(x)|\}$$

In the Hampel filter, each observation, x , will be compared to the median. In the case of the normal distribution, the Hampel filter is equivalent to the z-score, and epsilon can be chosen the same way as for the z-score.

Anomaly detection for Time-Series

Anomaly detection in time series data involves identifying unusual patterns or deviations from expected behavior.

1. Z-Score Method

identifies anomalies based on how far a data point deviates from the mean, measured in standard deviations.

```
from pandas import read_csv
from matplotlib import pyplot
data = pd.read_csv('SolieuThuTSA001_05.csv')

# Calculate Z-Scores:
mean = data['CPI_rate'].mean()
std_dev = data['CPI_rate'].std()
data['z_score'] = (data['CPI_rate'] - mean) / std_dev
```

Anomaly detection for Time-Series

```
# Identify Anomalies:  
threshold = 3 # Customize threshold  
anomalies = data[np.abs(data['z_score']) > threshold]  
print(anomalies)
```



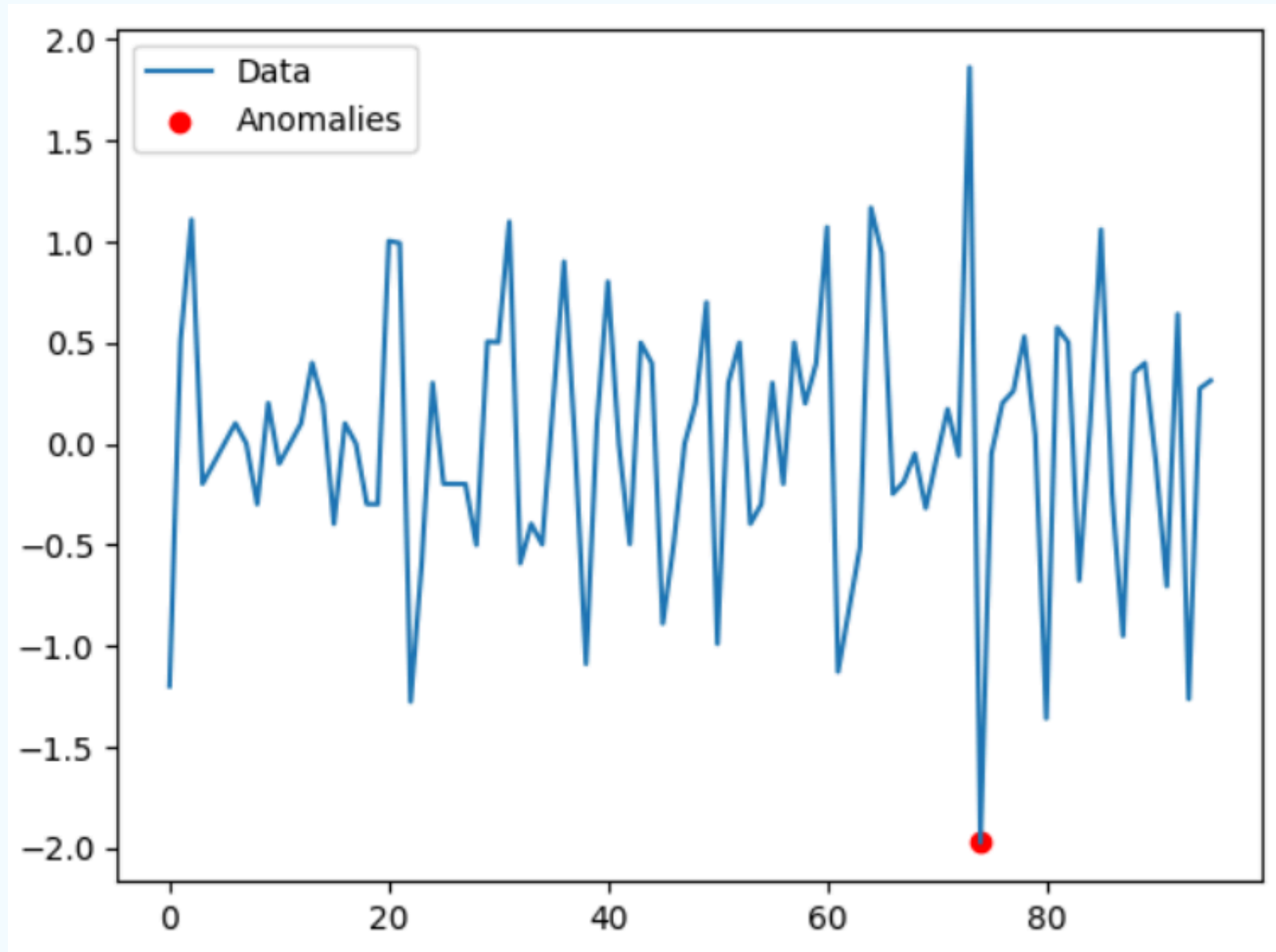
	Time	Year	Month	CPI	CPI_next	CPI_rate	z_score
74	75	2021	3	99.79	98.02554	-1.97446	-3.084322

Anomaly detection for Time-Series

```
# Visualize Results:
import matplotlib.pyplot as plt

plt.plot(data.index, data['CPI_rate'], label='Data')
plt.scatter(anomalies.index, anomalies['CPI_rate'],
            color='red', label='Anomalies')
plt.legend()
plt.show()
```

Anomaly detection for Time-Series



Anomaly detection for Time-Series

```
# Identify Anomalies:  
threshold = 2 # Customize threshold  
anomalies = data[np.abs(data['z_score']) > threshold]  
print(anomalies)
```

↗	Time	Year	Month	CPI	CPI_next	CPI_rate	z_score
73	74	2021	2	101.80	101.861117	1.861117	2.888107
74	75	2021	3	99.79	98.025540	-1.974460	-3.084322
80	81	2021	9	99.40	98.640468	-1.359532	-2.126809

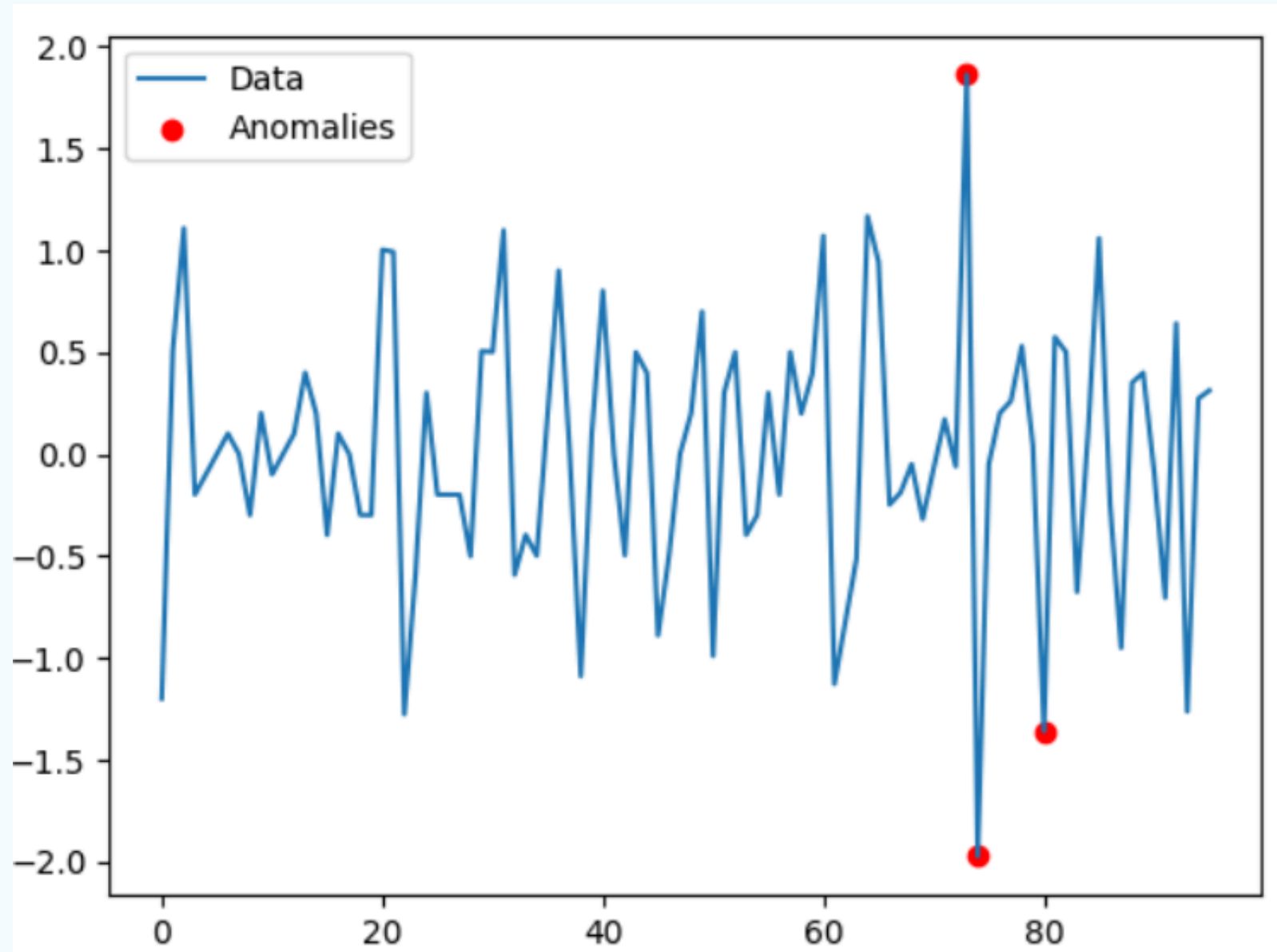
Anomaly detection for Time-Series

```
# Visualize Results:
```

```
import matplotlib.pyplot as plt
```

```
plt.plot(data.index, data['CPI'], label='Data')  
plt.scatter(anomalies.index, anomalies['CPI'],  
            color='red', label='Anomalies')  
plt.legend()  
plt.show()
```

Anomaly detection for Time-Series



Anomaly detection for Time-Series

2. Isolation Forest

This machine learning method isolates anomalies by randomly partitioning the data.

- **Import and Prepare Data:**

```
from sklearn.ensemble import IsolationForest
```

```
X = data[['CPI_rate']].values
```

- **Train Isolation Forest:**

```
model = IsolationForest(contamination=0.05,  
random_state=42)  
data['anomaly'] = model.fit_predict(X)
```

Anomaly detection for Time-Series

- **Filter Anomalies:**

```
anomalies = data[data['anomaly'] == -1]
print(anomalies)
```

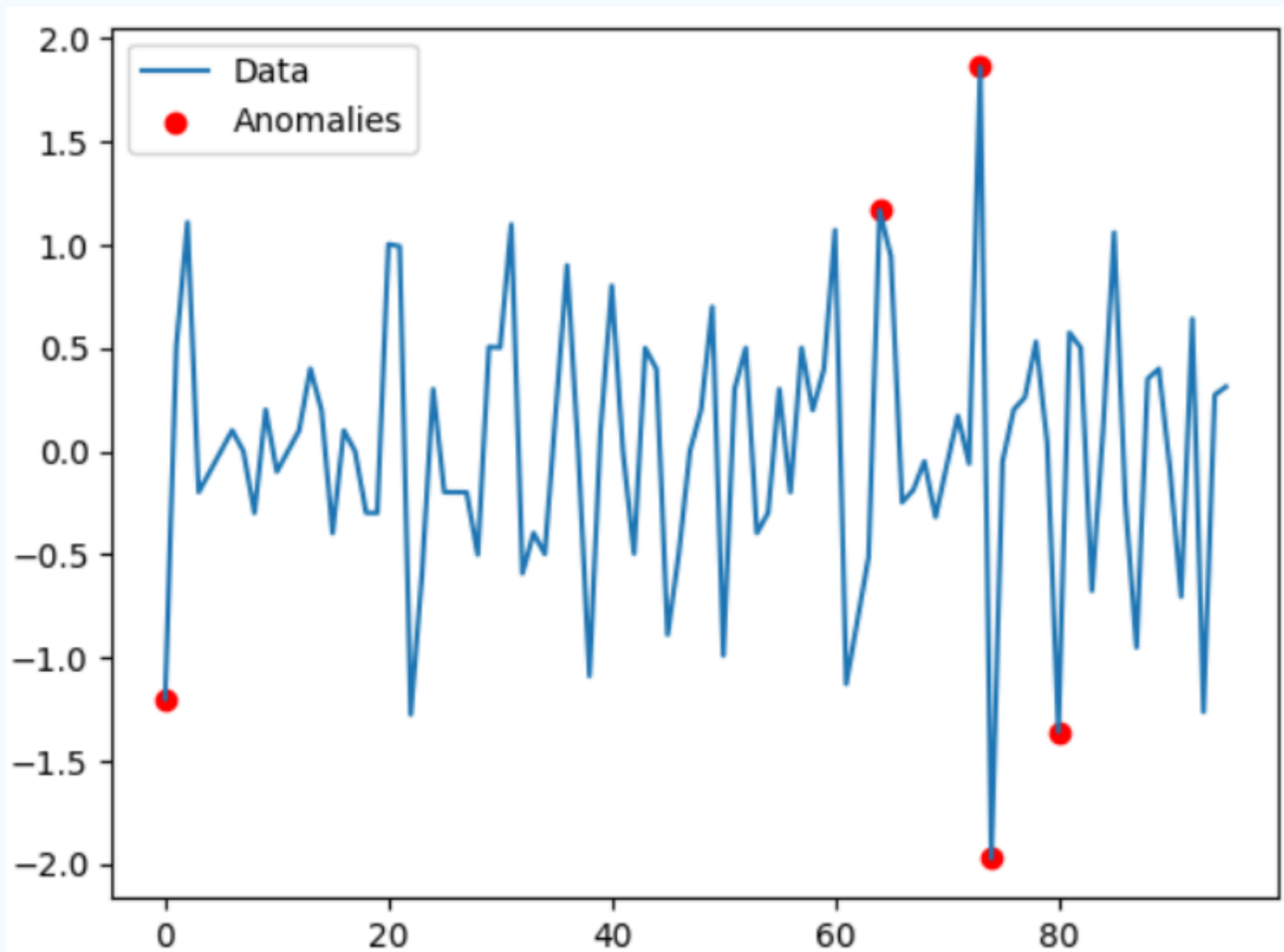


	Time	Year	Month	CPI	CPI_next	CPI_rate	z_score	anomaly
0	1	2015	1	98.80	98.800000	-1.200000	-1.878400	-1
64	65	2020	5	99.75	101.166329	1.166329	1.806243	-1
73	74	2021	2	101.80	101.861117	1.861117	2.888107	-1
74	75	2021	3	99.79	98.025540	-1.974460	-3.084322	-1
80	81	2021	9	99.40	98.640468	-1.359532	-2.126809	-1

- **Visualize Results:**

```
plt.plot(data.index, data['CPI_rate'], label='Data')
plt.scatter(anomalies.index, anomalies['CPI_rate'],
            color='red', label='Anomalies')
plt.legend()
plt.show()
```

Anomaly detection for Time-Series



Change Point detection for Time-Series

[Ruptures: Change Point Detection in Python](#)

Ruptures is a Python library designed for **off-line change point detection**, which is the process of identifying points in a signal where statistical properties change. It is particularly useful for analyzing and segmenting **non-stationary signals**. The library supports both **parametric** and **non-parametric** models, offering a variety of exact and approximate algorithms.

```
pip install ruptures
```

Change Point detection for Time-Series

```
pip install ruptures
import ruptures as rpt

from google.colab import files
import pandas as pd
import io
import numpy as np
uploaded = files.upload()
```



Choose Files DongNamA_GDP_C.csv

DongNamA_GDP_C.csv(text/csv) - 4604 bytes, last modified: 10/10/2025 - 100% done
Saving DongNamA_GDP_C.csv to DongNamA_GDP_C.csv

```
from pandas import read_csv
from matplotlib import pyplot
data = pd.read_csv('DongNamA_GDP_C.csv')
```

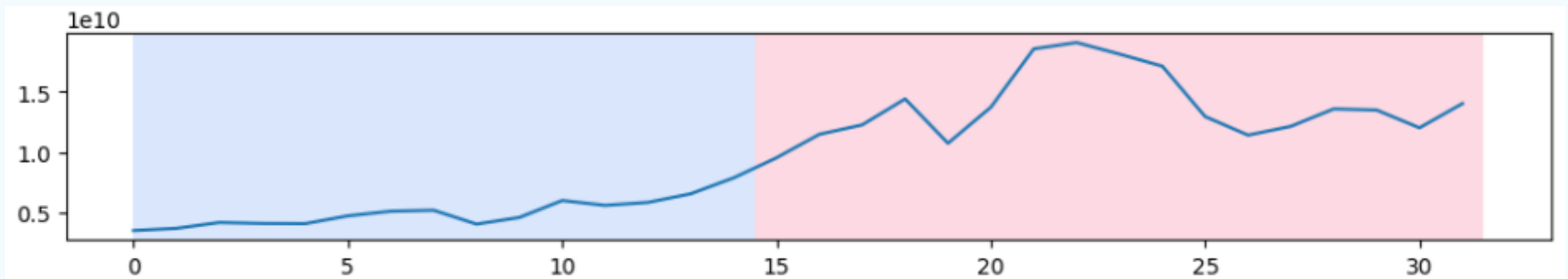
Change Point detection for Time-Series

```
X = data[['Brunei']].values
```

```
# Apply the Pelt algorithm for change point  
detection
```

```
algo = rpt.Pelt(model="rbf").fit(X)  
result = algo.predict(pen=10)
```

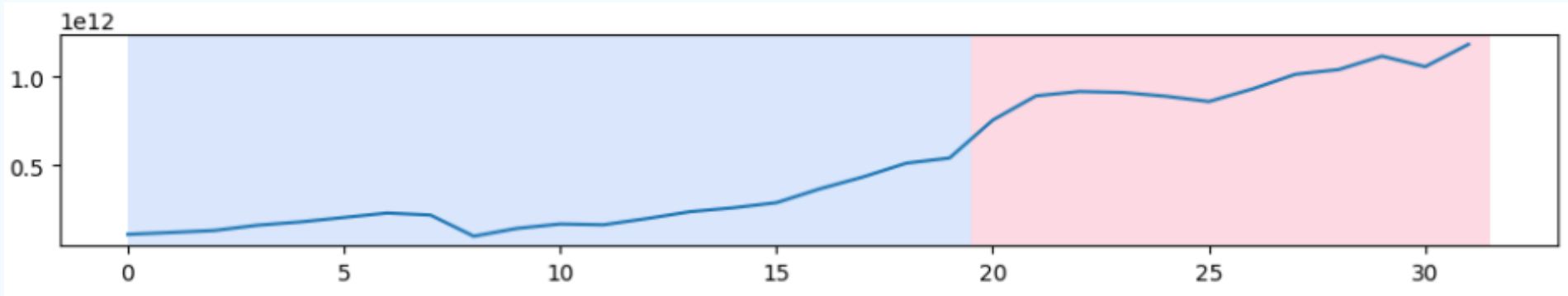
```
# Display the results  
rpt.display(X, result)  
plt.show()
```



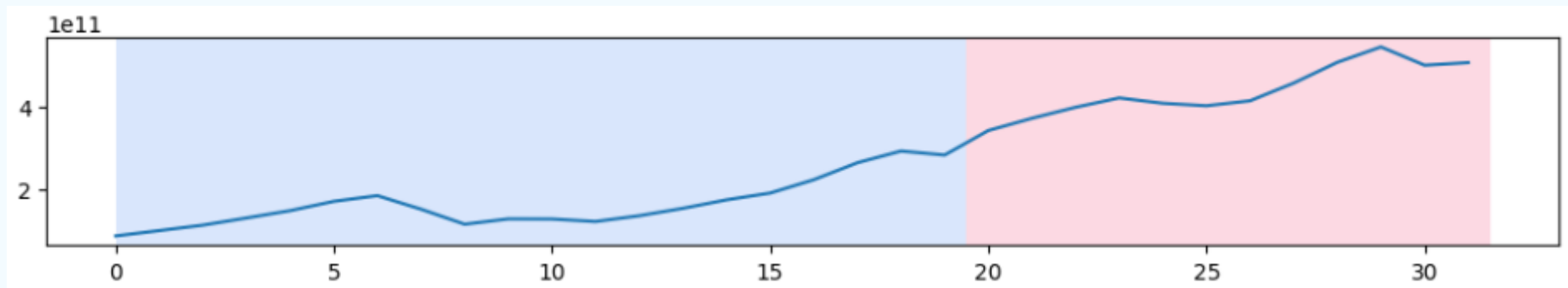
Change Point detection for Time-Series

```
X = data[['Indonesia']].values
```

```
• • •
```

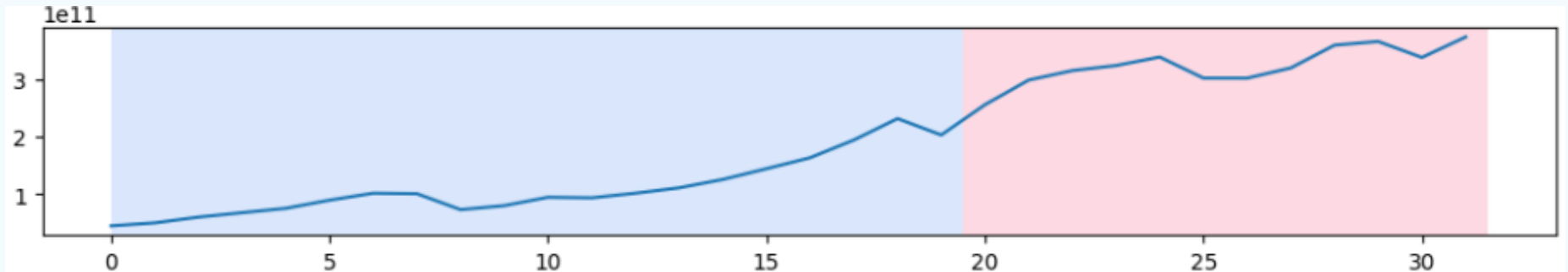


```
X = data[['Thailand']].values
```

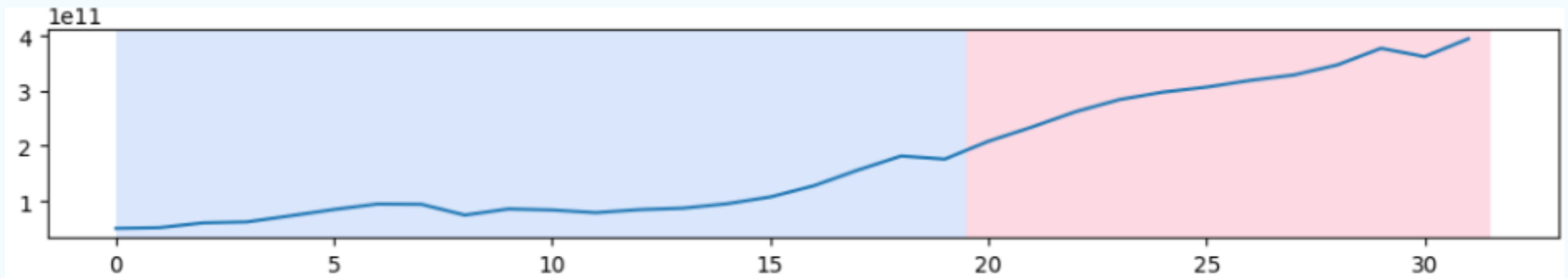


Change Point detection for Time-Series

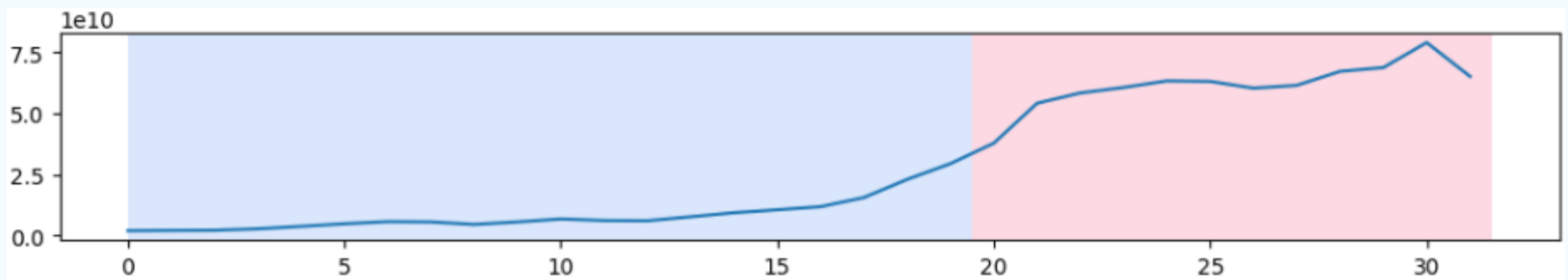
```
X = data[['Malaysia']].values
```



```
X = data[['Philippines']].values
```

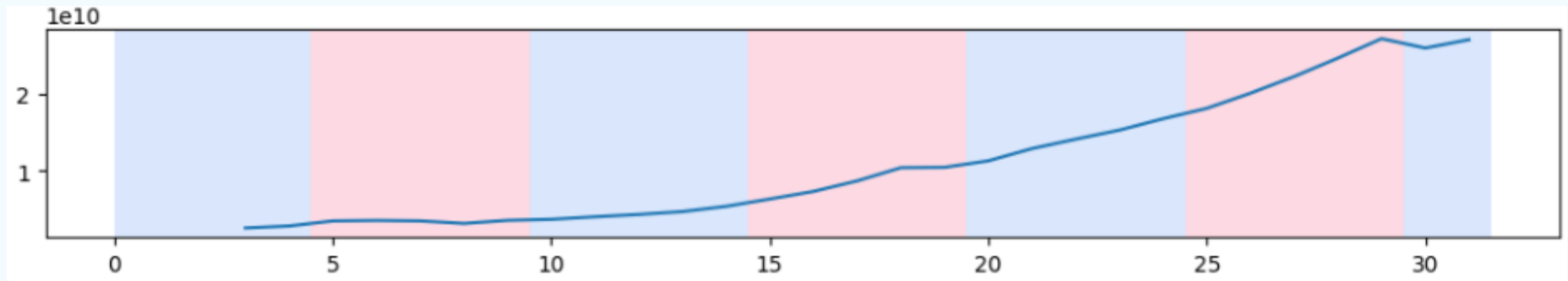


```
X = data[['Myanmar']].values
```



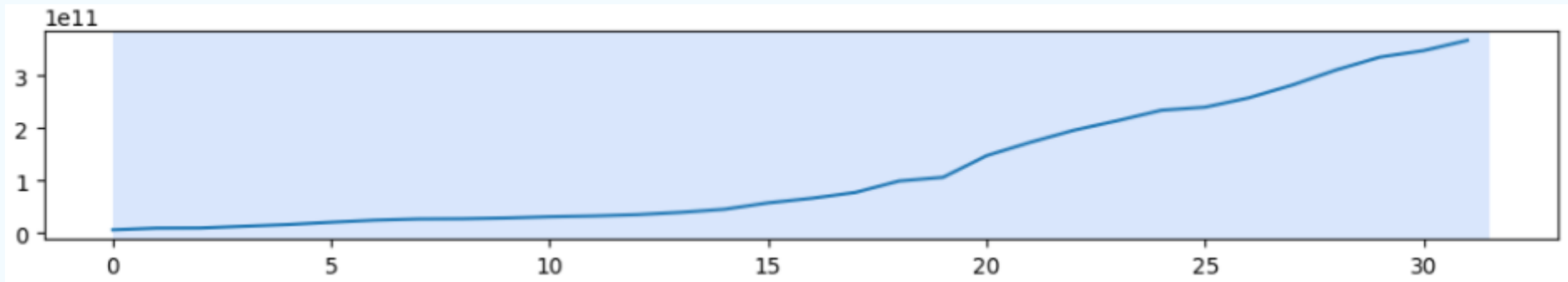
Change Point detection for Time-Series

```
X = data[['Cambodia']].values
```

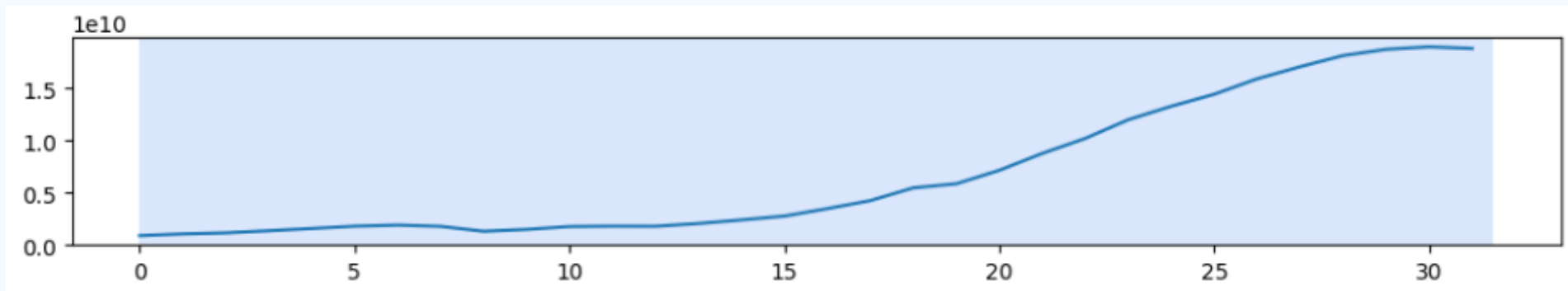


Change Point detection for Time-Series

```
X = data[['Vietnam']].values
```



```
X = data[['LaoPDR']].values
```



THANK YOU